プログラミング３ 講義資料

第7回 GitHub Issue・Pull Request

- Issue による作業管理
- Pull Request とコードレビュー
- GitHub Flow の基本

目次

- Issue
- Pull Request とレビュー

Issue

今回の目的と到達目標

今回の目的

- Issue で作業を管理できる
- PR の流れを理解する
- レビュー観点を掴む

到達目標

- [2] Issue/PR を活用したチーム開発のワークフロー

Issue とは

- GitHub 上の **作業単位・議論の入れ物**
- 「やること」「バグ」「疑問」をチームで共有する
- ラベル・担当者・マイルストーンで整理できる

要素	役割
タイトル	一目で内容が分かる短い要約
本文	背景・要件・受け入れ条件
ラベル	bug / feature / doc など分類
担当者	誰が進めるか

良い Issue の書き方

1. **タイトルは動詞で始める**: 「ログイン画面を実装する」
2. **背景を書く**: なぜ必要か、関連する機能は何か
3. **受け入れ条件**: 何ができたら完了か（チェックリスト）
4. **関連情報**: スクショ・参考URL・関連 Issue 番号（`#12`）

> チームが同じ目標に向かえるかは Issue の質で決まる

Pull Request とレビュー

Pull Request の役割

- 「このブランチを main に取り込んでほしい」という依頼
- 差分・コミット履歴・議論を1箇所に集約
- マージ前に **他メンバーの目が入る** ことでバグ・設計ミスを早期発見

main ← PR ← feature/foo

PR 本文に含めること
対応する Issue 番号 (`closes #12`)
変更の概要と理由
動作確認の方法

GitHub Flow の基本形

1. `main` から feature ブランチを切る
2. 変更を commit・push
3. GitHub で Pull Request を作成
4. レビュー → 修正コミット追加
5. Approve → merge
6. feature ブランチを削除

> 本講義のチーム開発は main への直接 push を **禁止**、必ず PR 経由

レビュー観点

観点	何を見る
正しく動くか	動作確認の記録があるか
読みやすいか	命名・構造・コメント
影響範囲	他機能を壊していないか
テスト	受け入れ条件を満たすか
ドキュメント	README・仕様書に反映されたか

- 建設的なコメントを: **変更提案** と **質問** を区別する

レビュワーのマナー

Tips

- 「〜した方がよいのでは？」と **提案** として書く
- 人格ではなく **コード** を論じる（良い: 「ここは X が読みやすい」）
- 些細な指摘は `nit:` プレフィックスで温度を伝える
- 疑問は質問として聞く（決めつけない）
- approve は責任 — 動作確認したか？を自分に問う

今回のまとめ

- Issue は作業・議論の入れ物、タイトル+受け入れ条件を明確に書く
- PR は main への取り込み依頼、GitHub Flow が基本
- レビューは建設的に、コードを論じて人を論じない

今回カバーした MCC 項目

- V-D-1 プログラミング

次回予告

- 第8回: チーム開発準備 — テーマ設定・最終レポート案内